Day 01 Exercises

Exercise 1 – Type Deduction with `auto`
Write a program that declares four variables using `auto` with the initializers `42`, `42.0`, `{1}` (with equals), and `{1}` (without equals). Print the size of each variable and use `typeid` to display the deduced type.
Hints: `auto`, `std::initializer_list`, `typeid`, `sizeof`.

Exercise 2 – Range-Based `for` Loop: Copy vs. Reference
Write a program that creates a `std::vector` of `Student` structs (with `name` and `score`). Iterate over the vector twice: first using `for (auto student : students)` and modify the local copy's score; then using `for (const auto& student : students)` and print each student. Show that the original data is unchanged.
Hints: range-based `for`, `auto`, `const auto&`, `std::vector`.

Exercise 3 – `nullptr` vs. `0`
Write a program with two overloaded functions: `void show(int)` and `void show(const char*)`. Call `show(0)` and `show(nullptr)`. Observe which overload is selected and explain the output.
Hints: `nullptr`, function overloading, `std::cout`.

Exercise 4 – Narrowing Conversion
Write a program that attempts to initialize an `int` with `int quantity{2.75};`. Record the compiler error. Then modify the program to use `static_cast<int>(2.75)` and print the result.
Hints: brace initialization, narrowing conversion, `static_cast`.

Exercise 5 – Scoped Enumeration and `switch`
Write a program that defines `enum class FileState { missing, opened, processed, failed };` and a function `void printState(FileState state)` that uses a `switch` to print a message for each state. Call the function with each enumerator.
Hints: `enum class`, `switch`, scoped enumerators.

Exercise 6 – `if` with Initializer
Write a program that uses a `std::map<std::string, std::string>` and looks up a key. Use `if (const auto it = map.find(key); it != map.end())` to print the value if found, or a "not found" message otherwise. Observe that the iterator's scope ends with the `if`.
Hints: `if` initializer, iterator, `std::map::find`.